

Linked Lists

A linked list is a `Link` object or `Link.empty`.

You can mutate a `Link` object `s` in two ways:

- Change the first element with `s.first = ...`
- Change the rest of the elements with `s.rest = ...`

```
class Link:
    """A linked list is either a Link object or Link.empty

    >>> s = Link(3, Link(4, Link(5)))
    >>> s.rest
    Link(4, Link(5))
    >>> s.rest.rest.rest is Link.empty
    True
    >>> s.rest.first * 2
    8
    >>> print(s)
    (3 4 5)
    """
    empty = ()

    def __init__(self, first, rest=empty):
        assert rest is Link.empty or isinstance(rest, Link)
        self.first = first
        self.rest = rest

    def __repr__(self):
        if self.rest:
            rest_repr = ', ' + repr(self.rest)
        else:
            rest_repr = ''
        return 'Link(' + repr(self.first) + rest_repr + ')'

    def __str__(self):
        string = '('
        while self.rest is not Link.empty:
            string += str(self.first) + ' '
            self = self.rest
        return string + str(self.first) + ')'
```

Q1: Strange Loop

Here is a `Link` object with a cycle that represented an infinite repeating list of 1's. It may be useful as a reference doctest to help you solve the problem.

```
>>> ones = Link(1)
>>> ones.rest = ones
>>> [ones.first, ones.rest.first, ones.rest.rest.first, ones.rest.rest.rest.first]
[1, 1, 1, 1]
>>> ones.rest is ones
True
```

Implement `strange_loop`, which takes no arguments and returns a `Link` object `s` for which `s.rest.first.rest` is `s`.

Draw a picture of the linked list you want to create, then write code to create it.

```
def strange_loop() -> Link:
    """Return a Link s for which s.rest.first.rest is s.

    >>> s = strange_loop()
    >>> s.rest.first.rest is s
    True
    """
    """*** YOUR CODE HERE ***"""
```

Q2: Sum Two Ways

Implement both `sum_rec` and `sum_iter`. Each one takes a linked list of numbers `s` and a non-negative integer `k` and returns the sum of the first `k` elements of `s`. If there are fewer than `k` elements in `s`, all of them are summed. If `k` is 0 or `s` is empty, the sum is 0.

Use recursion to implement `sum_rec`. Don't use recursion to implement `sum_iter`; use a `while` loop instead.

To get started on the recursive implementation, consider the example `a = Link(1, Link(6, Link(8)))`, and the call `sum_rec(a, 2)`. Write down the recursive call to `sum_rec` that would help compute `sum_rec(a, 2)`. Then, write down what that recursive call should return. Discuss how this return value is useful in computing the return value of `sum_rec(a, 2)`.

```
def sum_rec(s: Link, k: int) -> int:
    """Return the sum of the first k elements in s.

    >>> a = Link(1, Link(6, Link(8)))
    >>> sum_rec(a, 2)
    7
    >>> sum_rec(a, 5)
    15
    >>> sum_rec(Link.empty, 1)
    0
    """
    # Use a recursive call to sum_rec; don't call sum_iter
    """*** YOUR CODE HERE ***"""
```

```
def sum_iter(s: Link, k: int) -> int:
    """Return the sum of the first k elements in s.

    >>> a = Link(1, Link(6, Link(8)))
    >>> sum_iter(a, 2)
    7
    >>> sum_iter(a, 5)
    15
    >>> sum_iter(Link.empty, 1)
    0
    """
    # Don't call sum_rec or sum_iter
    """*** YOUR CODE HERE ***"""
```

Q3: Overlap

Implement `overlap`, which takes two linked lists of numbers called `s` and `t` that are sorted in increasing order and have no repeated elements within each list. It returns the count of how many numbers appear in both lists.

This can be done in *linear* time in the combined length of `s` and `t` by always advancing forward in the linked list whose first element is smallest until both first elements are equal (add one to the count and advance both) or one list is empty (time to return).

```
def overlap(s: Link, t: Link) -> int:
    """For increasing s and t, count the numbers that appear in both.

    >>> a = Link(3, Link(4, Link(6, Link(7, Link(9, Link(10))))))
    >>> b = Link(1, Link(3, Link(5, Link(7, Link(8))))
    >>> overlap(a, b)  # 3 and 7
    2
    >>> overlap(a.rest, b)  # just 7
    1
    >>> overlap(Link(0, a), Link(0, b))
    3
    """
    "*** YOUR CODE HERE ***"
```

Q4: Iterate in Order

Implement `iterate_in_order`, which takes two linked lists of numbers called `s` and `t` that are sorted in increasing order and have no repeated elements within each list. It returns a generator that iterates over all items in `s` and `t` in non-decreasing order.

```
def iterate_in_order(s: Link, t: Link):
    """For increasing s and t, yields the elements in s and t, in non-decreasing order.

    >>> a = Link(3, Link(4, Link(6, Link(7, Link(9, Link(10)))))
    >>> b = Link(1, Link(3, Link(5, Link(7, Link(8))))
    >>> t = iterate_in_order(a, b)
    >>> for item in t:
    ...     print(item)
    1
    3
    3
    4
    5
    6
    7
    7
    8
    9
    10
    >>> t = iterate_in_order(Link.empty, b)
    >>> for item in t:
    ...     print(item)
    1
    3
    5
    7
    8
    """
    "*** YOUR CODE HERE ***"
```

Efficiency

Tips for finding the order of growth of a function's runtime:

- If the function is recursive, determine the number of recursive calls and the runtime of each recursive call.
- If the function is iterative, determine the number of inner loops and the runtime of each loop.
- Ignore coefficients. A function that performs n operations and a function that performs $100 * n$ operations are both linear.
- Choose the largest order of growth. If the first part of a function has a linear runtime and the second part has a quadratic runtime, the overall function has a quadratic runtime.

Q5: The First Order...of Growth

What is the efficiency of `rey`?

```
def rey(finn):
    poe = 0
    while finn >= 2:
        poe += finn
        finn = finn / 2
    return
```

Choose one of:

- Constant
- Logarithmic
- Linear
- Quadratic
- Exponential
- None of these

What is the efficiency of `mod_7`?

```
def mod_7(n):
    if n % 7 == 0:
        return 0
    else:
        return 1 + mod_7(n - 1)
```

Choose one of:

- Constant
- Logarithmic
- Linear
- Quadratic
- Exponential
- None of these

Q6: Efficiency Practice

Choose the term that fills in the blank for the functions defined below: <function> runs in ____ time in the length of its input.

- Constant
- Logarithmic
- Linear
- Quadratic
- Exponential
- None of these

Assume that `len` runs in constant time and `all` runs in linear time in the length of its input. Selecting an element of a list by its index requires constant time. Constructing a range requires constant time.

```
def count_partitions(n, m):
    """Counts the number of partitions of a positive integer n,
    using parts up to size m."""
    if n == 0:
        return 1
    elif n < 0:
        return 0
    elif m == 0:
        return 0
    else:
        with_m = count_partitions(n-m, m)
        without_m = count_partitions(n, m-1)
        return with_m + without_m

def is_palindrome(s):
    """Return whether a list of numbers s is a palindrome."""
    return all([s[i] == s[len(s) - i - 1] for i in range(len(s))])

def binary_search(lst, n):
    """Takes in a sorted list lst and returns the index where integer n
    is contained in lst. Returns -1 if n does not exist in lst."""
    low = 0
    high = len(lst)
    while low <= high:
        middle = (low + high) // 2
        if lst[middle] == n:
            return middle
        elif n < lst[middle]:
            high = middle - 1
        else:
            low = middle + 1
    return -1
```

The `is_palindrome` question was reformatted from question 6(d) on fall 2019's [final](#).